

# ICCAD Contest Problem A - Hardware Trojan Detection on Gate-Level Netlist

Chung-Han Chou, Hung-Chun Chiu, Chih-Jen (Jacky) Hsu, and Kuan-Hua Tu  
Cadence Design Systems, Inc.

- 2025.07.29 Custom Docker Command
- 2025.05.20 Environment Setup
- 2025.02.24 First Version

## 1. Introduction

In recent years, the integrated circuit (IC) industry has experienced rapid growth and transformation. The development of complex systems-on-chip (SoCs) has been greatly facilitated by the use of third-party intellectual property (IP) cores. These IP cores are essential in meeting the increasing demands for functionality and performance in modern IC designs. However, this trend has also introduced significant security concerns. The reliance on third-party IPs can expose IC designs to vulnerabilities, as malicious actors may exploit these components to implant hardware Trojans (HTs), especially for sensitive applications.

Hardware Trojans (HTs) are malicious modifications to integrated circuits that can compromise security, potentially leading to unauthorized access, data leakage, or system failures. This issue is particularly critical in sensitive sectors such as military, financial, and governmental infrastructures. Traditional detection methods often rely on comparing the suspect circuit to a "golden" chip known to be Trojan-free, utilizing side-channel measurements such as power consumption or delay. However, obtaining a golden chip is often impractical due to cost and availability constraints. This limitation has highlighted the need for innovative detection methods.

Recent advancements in Machine Learning (ML) have shown significant potential across various fields. By extracting patterns and features extracted from the netlists, ML approaches can potentially eliminate the dependency on a golden chip. Some research has proposed that ML-assisted HT detection methods could achieve high accuracy and reliability for enhancing the security of ICs against HT threats.

In this contest, we define specific Hardware Trojan behaviors and challenge the contestants to develop programs that analyze given gate-level netlists to determine whether they contain Hardware Trojans. Participants are also encouraged to identify and mark specific Trojan gates for additional evaluation.

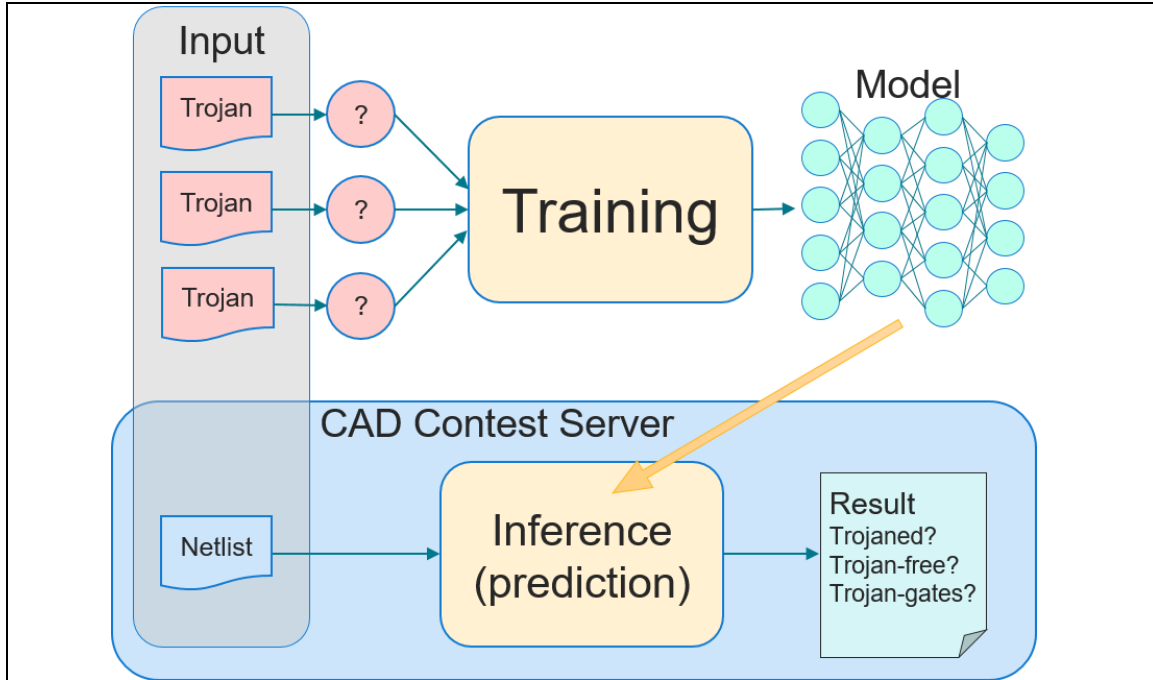


Figure 1. The expected framework.

## 2. Background

### 2.1 Hardware Trojan Detection

[1] surveys various methods for addressing Hardware Trojan (HT) detection. In the research by [2], a classification method using Support Vector Machine (SVM) and Neural Networks (NN) was proposed to identify HTs at the gate level. This method effectively detects HT nets by extracting and analyzing specific features, but it may struggle with scalability for larger designs. [3] introduced a reverse engineering-based approach using One-class SVM, which avoids the need for netlist extraction by directly analyzing IC images. This method simplifies the detection process but may miss subtle HTs embedded deeply within the design. [4] proposed the COTD method, which eliminates the need for a golden model by using controllability and observability metrics to detect HTs. While this approach reduces dependency on golden ICs, it may face challenges in accurately classifying HTs under varying process conditions. [5] developed the LASCA method, leveraging neural networks to correlate static timing data with delay information for HT detection. This method effectively mitigates the impact of voltage noise and process variations but requires extensive training data. The HERO framework by [6] employs a holistic approach using Deep Neural Networks (DNN) to analyze features from different manufacturing stages, providing a comprehensive detection mechanism. However, the complexity of integrating various data sources can be a drawback. [7][8] utilized Graph Neural Networks (GNN) to represent hardware designs and detect HTs, offering a robust solution for complex designs but requiring significant computational resources. Lastly, [8] proposed AdaTest, an adaptive test pattern generation framework using Reinforcement Learning (RL)

to prioritize informative test samples. This method enhances detection accuracy but may be computationally intensive for large-scale circuits.

### 3. Problem Formulation and Input Output Format

This section defines the input, output, and requirements of this contest.

First, there are some Trojans RTL will be provided to define the behavior of Trojan. However, this Trojans RTL will not be taken as input of the submitted executable. The contestants can perform synthesis or any extraction to get the “features” from the Trojans.

After that, the submitted executable file by the contestants should take one input netlist, which is a flattened Verilog netlist to be detected, and output the result to the output file. Contestants need to write a program to detect if the given netlist is Trojan\_Free or Trojaned. If the netlist is Trojaned, the contest should also output the Trojan gates to the output file.

#### 3.1 Program requirement

The developed program must be run on a Linux system. The time limit for each case is 1800 seconds. Parallel computation with multiple threads and GPU is allowed.

The program should accept two groups of arguments, “-netlist <netlist\_path/name.v>” and “-output <output\_path/name.v>”, for specifying the input and output files, respectively. For example, the command line for executing the program can be “./cada0000\_alpha -netlist design.v -output result.txt”. Note that, the contestant should change “0000” to their team ID. The name of the executable will change to cada0000\_beta or cada0000\_final when the contestant submits to beta test or final evaluation.

#### 3.2 Input file format

- The input netlist is a flattened netlist in Verilog format without hierarchy (one top module only).
- The netlist is composed of:
  1. Primitive gates (**and**, **or**, **nand**, **nor**, **not**, **buf**, **xor**, **xnor**)
  2. Flip-flops: dff as in figure 2
  3. Wires
  4. Constant values (1'b1, 1'b0)

Note that all primitive gates are assumed to have only 2 inputs and 1 output except for **buf** and **not** gates, which have only 1 input and 1 output.

```
module dff (  
    input wire clk,      // Clock input  
    input wire rst_n,    // Asynchronous active-low reset  
    input wire d,        // Data input  
    output reg q         // Data output  
);  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            q <= 1'b0;  
        else
```

```

q <= d;
end
endmodule

```

Figure 2. The expected optimization framework.

### 3.3 Output file format and requirements

- The first line is a string to indicate the netlist is Trojaned or not. For a Trojaned input, the first line will be "TROJANED". Otherwise, the first line will be "NO\_TROJAN"
- For Trojaned netlists, please print "TROJAN\_GATES" in the next line, and list the Trojan gate one by one. Print the keyword "END\_TROJAN\_GATES" in the end of Trojan gates.
- Figure 3 shows an example of an output.

```

1 TROJANED
2 TROJAN_GATES
3 g01
4 g02
5 g03
6 g09
7 g10
8 END_TROJAN_GATES

```

(a) Trojaned output

```

1 NO_TROJAN

```

(b) Trojan-Free output

Figure 3. The expected optimization framework.

## 4. Evaluation Criteria

- Correctness is necessary. The contestants will get zero points for the case if marking a Trojaned netlist with NO\_TROJAN or making a Trojan-free netlist as TROJANED.
- For Trojaned cases, we use F1 score to evaluate the Trojan gates marked by the contestants. The F1 score is defined as following:

| Estimate\ Fact  | Trojan Gate         | Not Trojan Gate     |
|-----------------|---------------------|---------------------|
| Trojan Gate     | True Positive (TP)  | False Positive (FP) |
| Not Trojan Gate | False Negative (FN) | True Negative (TN)  |

First, the number of TP, FP, and FN are calculated. Then, we define the Precision and Recall as following:

$$\textbf{Precision} = \frac{TP}{TP + FP}, \textbf{Recall} = \frac{TP}{TP + FN}$$

Finally, the F1 score is defined as

$$\textbf{F1} = 2 \times \frac{\textbf{Precision} \times \textbf{Recall}}{\textbf{Precision} + \textbf{Recall}}$$

- **Point** for each case: the contestants get 2 points if marking correctly. If the case is a Trojaned case, the contestants can get additional points equals to the evaluated **F1** score.
- **final\_score** is the sum of **point** of all cases.

- The team having the largest ***final\_score*** wins.

## 5. Examples

This section demonstrates a Trojan design that leaks the secret key from a cryptographic chip executing the Advanced Encryption Standard (AES) algorithm through a covert channel.

The Trojan design comprises two main modules: the top module, which integrates the AES-128 encryption and the Trojan circuit (TSC), which oversees the leakage process. The top module defines the overall structure, while the TSC module manages the pseudo-random number generator (PRNG) and the XOR modulation of the secret key bits.

```
module top (
    input wire clk,           // Clock input
    input wire rst,           // Asynchronous active-high reset
    input wire [127:0] state, // Data input
    input wire [127:0] key,   // Data input
    output wire [127:0] out   // Data output
    output wire [63:0] capacitance // Data output
);

    aes_128 AES (clk, state, key, out);
    TSC Trojan (rst, clk, key, capacitance);

endmodule
```

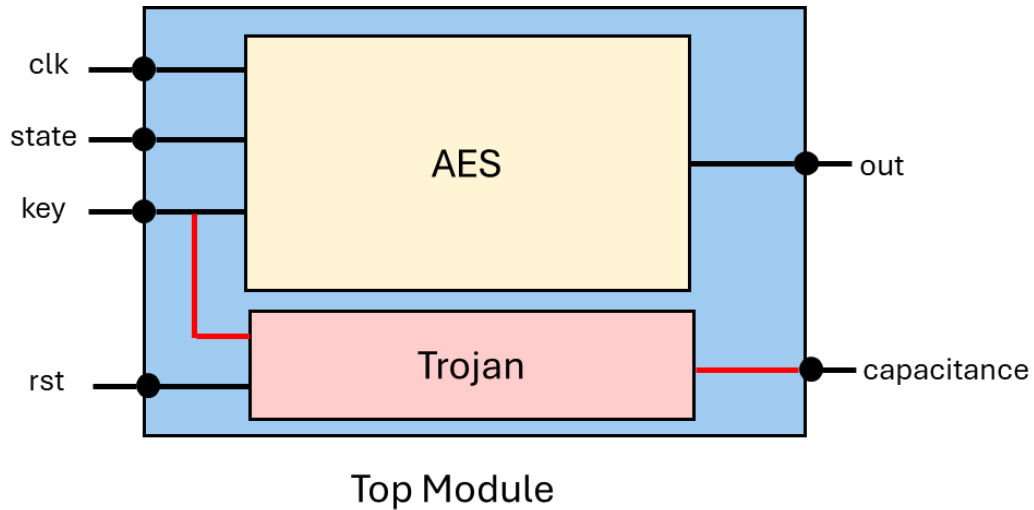
```
module TSC (
    input wire clk,           // Clock input
    input wire rst,           // Asynchronous active-high reset
    input wire [127:0] key,   // Data input
    output reg [127:0] load   // Data output
);
    wire [19:0] counter;
    lfsr_counter lfsr (rst, clk, counter);

    always @(posedge clk) begin
        integer i;
        for (i = 0; i < 64; i = i + 1) begin
            load[i] <= key[i / 8] ^ counter[i / 8];
        end
    end

endmodule
```

After the design is synthesized into the flatten netlist, contestants are required to identify the primitive gates that belong to the Trojan. Specifically, for this example, contestants need to list all the primitive gates within the TSC module.

The block diagram below illustrates the overall design of the AES Trojan. The red block represents the TSC Trojan, which contains primitive gates. Contestants need to locate all the primitive gates within the red block.



## 6. Evaluation Environment

All submissions will be executed within the Docker image [11] provided by the contestants on an Ubuntu 22.04 host with:

- CPU: 4 x Intel Xeon Platinum 8259CL CPU
- RAM: 16 GB
- GPU: 1 x NVIDIA Tesla T4 (16GB VRAM)
- Drivers: Nvidia 535.183.01
- Docker: 26.1.3 with NVIDIA Container Toolkit

We will load the docker image file with the following command:

```
$ docker load --input cada0000_alpha_image.tar.gz
```

Please ensure the program name is correct. Contestants must replace '0000' with their team ID. The program should be executed using the following command.

```
$ docker run --gpus all \  
  cada0000_alpha_image \  
  ./cada0000_alpha -netlist <docker_test_data_path> -output <docker_result_path>
```

(2025.07.29 Update) If customization is required, contestants must provide the appropriate docker run command with the necessary options.

For example, contestants may remove the `--gpus all` and add the `--platform linux/arm64` option and write the customized command into a file named `cada0000_custom_command.txt`. This file, along with the Docker image, should then be uploaded. Please ensure the custom command file name is correct. Contestants must replace '0000' with their team ID.

```
$ docker run --platform linux/arm64 \  
  cada0000_alpha_image \  
  ./cada0000_alpha -netlist <docker_test_data_path> -output <docker_result_path>  
cada0000_custom_command.txt.
```

The template Dockerfile is provided below. Contestants can modify it to suit their environment but must ensure the working directory is set to `/app`.

```
FROM nvidia/cuda:12.2.0-runtime-ubuntu22.04  
  
RUN apt-get update && apt-get install -y --no-install-recommends \  
  python3 \  
  python3-pip \  
  && rm -rf /var/lib/apt/lists/*  
  
RUN pip3 install torch numpy  
  
# Set the working directory to /app  
WORKDIR /app  
  
COPY cada0000_alpha.sh /app/cada0000_alpha  
COPY cada0000_alpha.py /app/cada0000_alpha.py  
  
RUN chmod +x /app/cada0000_alpha
```

## 7. References

- [1] K. I. Gubbi, B. Saber Latibari, A. Srikanth, T. Sheaves, S. A. Beheshti-Shirazi, S. M. PD, S. Rafatirad, A. Sasan, H. Homayoun, and S. Salehi, "Hardware trojan detection using machine learning: A tutorial," *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 3, pp. 1–26, 2023
- [2] Hasegawa, K., Yanagisawa, M., & Togawa, N. (2017). A hardware-Trojan classification method using machine learning at gate-level netlists based on Trojan features. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 100(7), 1427-1438.
- [3] Bao, C., Forte, D., & Srivastava, A. (2014). On application of one-class SVM to reverse engineering-based hardware Trojan detection. *Fifteenth International Symposium on Quality Electronic Design*, 47-54.

- [4] Salmani, H. (2017). COTD: Reference-free hardware Trojan detection and recovery based on controllability and observability in gate-level netlist. *IEEE Transactions on Information Forensics and Security*, 12(2), 338-350.
- [5] Vakil, A., Behnia, F., Mirzaeian, A., Homayoun, H., Karimi, N., & Sasan, A. (2020). LASCA: Learning assisted side channel delay analysis for hardware Trojan detection. 2020 21st International Symposium on Quality Electronic Design (ISQED), 40-45.
- [6] Moustakidis, S., Liakos, K., Georgakilas, G., Sketopoulos, N., Seimoglou, S., Karlsson, P., & Plessas, F. (2020). A novel holistic approach for hardware Trojan detection powered by deep learning (HERO). *Proc. ATTRACT*.
- [7] Yasaei, R., Chen, L., Yu, S.-Y., & Al Faruque, M. A. (2022). Hardware Trojan detection using graph neural networks. *arXiv preprint arXiv:2204.11431*.
- [8] Yasaei, R., Yu, S.-Y., & Al Faruque, M. A. (2021). GNN4TJ: Graph neural networks for hardware Trojan detection at register transfer level. 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE), 1504-1509.
- [9] Chen, H., Zhang, X., Huang, K., & Koushanfar, F. (2022). AdaTest: Reinforcement learning and adaptive sampling for on-chip hardware Trojan detection. *arXiv preprint arXiv:2204.06117*.
- [10] Gubbi, K. I., Saber Latibari, B., Srikanth, A., Sheaves, T., Beheshti-Shirazi, S. A., PD, S. M., Rafatirad, S., Sasan, A., Homayoun, H., & Salehi, S. (2023). Hardware Trojan Detection Using Machine Learning: A Tutorial. *ACM Transactions on Embedded Computing Systems*, 22(3), Article 46.
- [11] <https://www.docker.com/>